

Test Plan

IBA Campus Facility Booking System

Abdul Wasay Imran	27126
Saad Imam	27079
Muhammad Imad Raza	26953
Syed Muhammad Deebaj Haider Kazmi	26012

[Github Repository Link](#)

Contents

1	Introduction	2
1.1	Purpose	2
1.2	Background	2
1.3	Scope	2
2	Test Items	3
3	Scope of Testing	4
4	Test Objectives	5
5	Test Strategy and Approach	6
5.1	Test Techniques	6
5.2	Test Levels	6
5.3	Test Types	6
6	Risk Analysis	7
6.1	Known and Predictable Risk Categories	7
6.2	Product Risks vs. Project Risks	7
7	Test Schedule	9
8	Test Estimation	10
8.1	Phase 1 — Test Planning (Week 1)	10
8.2	Phase 2 — Test Design (Week 2)	10
8.3	Phase 3 — Environment Setup (Weeks 2–3)	10
8.4	Phase 4 — Test Execution (Weeks 3–4)	11
8.5	Phase 5 — Defect Fix & Regression (Weeks 4–5)	11
8.6	Phase 6 — UAT & Closure (Weeks 5–6)	11
9	Entry and Exit Criteria	12
9.1	Entry Criteria	12
9.2	Exit Criteria	12
10	Resources, Roles and Responsibilities	13
10.1	Team Structure	13
10.2	Test Environment and Tools	13
10.3	Defect Severity Classification	13
11	Test Deliverables	14
12	Test Cases	15
12.1	Authentication & Authorisation	15
12.2	Room Booking Request	16
12.3	Booking Management — Program Office	17
12.4	Booking Cancellation	17
12.5	Admin System Management	18
12.6	Data Integrity	18

1 Introduction

1.1 Purpose

This Test Plan defines the overall testing strategy, scope, objectives, resources, schedule, and test cases for the IBA Campus Facility Booking System. The document is the primary reference for all testing activities carried out by the test team as part of the Software Testing course project.

1.2 Background

The Institute of Business Administration (IBA) currently manages campus facility bookings (CFBS) through informal, manual methods including emails, physical registers, and verbal coordination. This leads to double-bookings, scheduling conflicts, and a lack of real-time visibility across the campus.

The platform addresses these issues by providing a centralised, web-based platform that supports three user roles: **Students**, **Program Office (PO)** staff, and **Administrators**. Students can request room bookings, the Program Office approves or rejects those requests, and Administrators manage the overall system configuration.

1.3 Scope

In scope:

- All nine feature areas from the SRS: Authentication, Room Booking, Booking Approval/Rejection, User Management (Admin), Room Management, Building Management, Booking Cancellation, Slot Blocking, and Real-time Availability
- Unit testing of individual backend controllers and service functions
- API-level integration testing of all REST endpoints
- End-to-end UI workflow testing via automated browser scripts
- Negative path testing, boundary value analysis, and equivalence partitioning
- Concurrency and conflict-detection testing (double booking, simultaneous slot requests)
- Role-based access control (RBAC) security testing
- Performance and load testing using free tools
- Data persistence and audit trail verification
- Frontend UX correctness: slot availability visibility, booking status indicators

Out of scope:

- Penetration testing beyond role-based access checks
- Production deployment and infrastructure testing

2 Test Items

The item under test is described in the SRS. Key details are summarised below.

- **System Name:** IBA Campus Facility Booking System
- **Version:** 1.0
- **Platform:** Web Application (React 18 + Vite + NestJS + Supabase PostgreSQL)
- **Frontend URL:** `http://localhost:5173`
- **Backend URL:** `http://localhost:3000/api`
- **Database:** Supabase PostgreSQL (dedicated test instance, separate from production)
- **Browsers Under Test:** Google Chrome (latest), Mozilla Firefox (latest)
- **Operating Systems:** Windows 10/11, CachyOS 6.19.12-1

3 Scope of Testing

All features specified in the SRS are within scope. The table below lists each feature with its priority, scope status, and the test techniques that will be applied. Priorities reflect both business impact and frequency of use.

Feature	Priority	Status	Test Techniques
Authentication (Student & PO Login)	High	In Scope	ECP, BVA, Error Guessing
Request Room Booking (Student)	High	In Scope	Decision Table, Use Case
Accept / Reject Room Booking (PO)	High	In Scope	Use Case, Decision Table, Error Guessing
Add Student (Admin)	High	In Scope	ECP, BVA
Add Program Office Member (Admin)	High	In Scope	ECP, BVA
Cancel Booking (Student / PO / Admin)	Medium	In Scope	Use Case
Add Room (Admin)	Medium	In Scope	ECP, Functional
Add Building (Admin)	Medium	In Scope	ECP, Functional
Disable Room (Admin)	Low	In Scope	State Transition, Functional

Table 1: Features to be Tested

4 Test Objectives

The testing effort for CFBS aims to achieve the following objectives:

1. **Verify Functional Correctness.** Confirm that all nine system features behave in accordance with the requirements defined in the SRS.
2. **Ensure Booking Conflict Prevention.** Verify that the system correctly rejects booking requests that conflict with an already-approved booking for the same room, date, and time period — including double-booking, blocked slots, and simultaneous requests from multiple students for the same slot.
3. **Validate Role-Based Access Control.** Ensure that Students, Program Office staff, and Administrators can only perform actions within their authorised scope. Unauthorised actions must be blocked with appropriate error messages.
4. **Confirm Booking Constraint Enforcement.** Validate that time slot restrictions (9 AM to 6 PM only), current-year date restrictions, and room-disabled states are properly enforced.
5. **Confirm Frontend Availability Visibility.** Verify that the frontend communicates slot availability clearly before a student submits a request, not only after a conflict error is returned.
6. **Validate PO Applicant Distinction.** Verify that the Program Office can distinguish between a student and a faculty/teacher applicant from the booking request details.
7. **Validate Input Handling and Edge Cases.** Confirm that edge cases — invalid credentials, past dates, zero capacity, duplicate ERP/username — are handled gracefully with clear, non-sensitive error messages.
8. **Assess User Notifications.** Confirm that the booking approval and rejection notification mechanism correctly informs the student of the Program Office decision.

Success Criterion: Testing is considered successful when all planned test cases have been executed, all Severity-1 (Critical) defects have been resolved and re-tested, and $\geq 90\%$ of Severity-2 (Major) defects are resolved prior to release.

5 Test Strategy and Approach

Testing for CFBS follows a risk-based, specification-driven strategy aligned with IEEE 829 and ISTQB principles. Higher-priority and higher-risk features authentication, room booking, and approval/rejection receive the most thorough coverage.

5.1 Test Techniques

Three families of test design techniques are applied:

- **Specification-Based**
- **Structure-Based**
- **Experience-Based**

5.2 Test Levels

Level	Scope
Unit Testing	Individual functions and components
Integration Testing	API endpoints, database interactions
System Testing	End-to-end functional scenarios
User Acceptance Testing	Real-user validation of all workflows

5.3 Test Types

- **Functional Testing** — primary type, applied to all in-scope features
- **Regression Testing** — re-executed after every defect fix
- **Negative Testing** — invalid inputs, unauthorised access attempts
- **Concurrency Testing** — simultaneous booking requests for the same slot
- **Performance Testing** — API response times under realistic data loads
- **Usability Testing** — informal assessment during UAT sessions

6 Risk Analysis

6.1 Known and Predictable Risk Categories

The following table applies known and predictable risk categories directly to the CFBS project. Each category is assessed for its specific impact, severity, and mitigation strategy.

Note: the severity rated below is relative to the project.

Risk Category	Impact on CFBS	Severity	Mitigation Strategy
Business Impact	Academic scheduling at IBA depends on correct bookings; errors affect all campus users	High	Obtain stakeholder sign-off on scope; provide regular status reports
Process Definition	No formal booking process existed before this system; process gaps may surface during testing	High	Strictly follow the test process defined in this plan; hold a process review before execution begins
Development Environment	The local development environment (PostgreSQL version, OS) may differ from the production setup	Medium	Perform environment parity checks before testing; use consistent Docker-based configuration
Technology to be Built	The React + NestJS + Supabase stack is relatively new to the project team	Medium	Conduct technology spike sessions; pair QA engineers with developers during integration testing

Table 2

6.2 Product Risks vs. Project Risks

Beyond the category-level analysis, specific risks are identified at two levels:

Product Risk	Project Risk
Double-booking defect allowing conflicting reservations	Test environment unavailable during scheduled execution
Booking time restrictions (9 AM–6 PM) not enforced	Scope changes to the SRS discovered after testing has begun
Cancelled bookings not correctly freeing room slots	Build delivered late, compressing the available test window
Concurrent booking requests bypassing conflict detection	Frontend not reflecting real-time slot availability

7 Test Schedule

The following six-week schedule covers the full testing lifecycle from planning through closure.

Phase	Activity	Duration	Key Tasks
1	Test Planning	Week 1	Review SRS; identify test conditions; finalise this test plan; define entry and exit criteria
2	Test Design	Week 2	Design test cases using ECP, BVA, Decision Tables, State Transition, and Use Case techniques
3	Environment Setup	Week 2–3	Configure test database; seed test data; set up Selenium and Postman collections
4	Test Execution	Week 3–4	Execute all planned test cases across all modules; log all defects
5	Defect Fix & Regression	Week 4–5	Developer fixes defects; QA re-runs regression suite; verify all resolutions
6	UAT & Closure	Week 5–6	User acceptance testing with representative users; obtain stakeholder sign-off; archive artefacts

8 Test Estimation

8.1 Phase 1 — Test Planning (Week 1)

Work Package	Story Points
Review SRS and identify all testable features	6
Define entry and exit criteria	3
Identify and document risk categories	5
Draft and finalise Master Test Plan document	13
Define defect severity classification scheme	1
Peer review of test plan	2

Table 3: Phase 1 — Test Planning Work Packages

8.2 Phase 2 — Test Design (Week 2)

Work Package	Story Points
Design auth test cases	5
Design booking request test cases	8
Design PO approval/rejection test cases (Use Case, Decision Table)	5
Design cancellation test cases (State Transition, BVA)	3
Design admin management test cases (ECP, BVA)	3
Design performance and data integrity test cases	3
Document all test cases	5

Table 4: Phase 2 — Test Design Work Packages

8.3 Phase 3 — Environment Setup (Weeks 2–3)

Work Package	Story Points
Configure Supabase dedicated test instance and isolate from production	5
Seed test database: student accounts, PO accounts, rooms, buildings	5
Set up NestJS backend in test environment and verify all API routes	3
Configure and verify React/Vite frontend against test backend	3
Set up Postman collections for all NestJS REST endpoints	5
Smoke-test the full environment (login, booking, PO flow) end-to-end	3

Table 5: Phase 3 — Environment Setup Work Packages

8.4 Phase 4 — Test Execution (Weeks 3–4)

Work Package	Story Points
Execute authentication test cases	8
Execute room booking test cases	8
Execute PO related test cases (PO approval, rejection, conflict resolution)	5
Execute cancellation test cases	5
Execute Admin test cases(admin user, room, and building management)	5
Log all defects with severity, steps to reproduce, and evidence	3

Table 6: Phase 4 — Test Execution Work Packages

8.5 Phase 5 — Defect Fix & Regression (Weeks 4–5)

Work Package	Story Points
Prioritise open defects with developer	3
Re-test fixed Severity-1 and Severity-2 defects	8
Execute full regression suite after each fix build	8
Update defect status and reconfirm closure	2

Table 7: Phase 5 — Defect Fix & Regression Work Packages

8.6 Phase 6 — UAT & Closure (Weeks 5–6)

Work Package	Story Points
Prepare UAT scripts and test data for representative users	5
Facilitate UAT sessions with student, PO, and admin participants	8
Record UAT observations and obtain stakeholder sign-off	3
Compile Test Execution Report (pass/fail metrics per module)	3
Write final Test Summary Report (coverage, defect metrics, release recommendation)	5
Archive all test artefacts (TestRail exports, JIRA snapshots, Postman collections)	2

Table 8: Phase 6 — UAT & Closure Work Packages

9 Entry and Exit Criteria

9.1 Entry Criteria

Testing of each module may begin only when all of the following conditions are satisfied:

- The SRS has been reviewed by the team
- The feature under test is code-complete and deployed to the test environment
- Test data (user accounts, rooms, buildings) has been seeded in the test database
- The test environment (frontend, backend, database) has been verified as operational
- All Severity-1 (Critical) defects from the preceding test cycle have been resolved

9.2 Exit Criteria

Testing may be formally concluded when:

- All planned test cases have been executed
- All Severity-1 (Critical) defects have been fixed, re-tested, and closed
- $\geq 90\%$ of Severity-2 (Major) defects have been resolved
- Test coverage is $\geq 95\%$ for all in-scope features
- UAT sign-off has been received from the project stakeholder

10 Resources, Roles and Responsibilities

10.1 Team Structure

Tester	ERP	Primary Responsibilities
Syed Muhammad Deebaj Haider Kazmi	26012	Test Lead coordinate execution, maintain traceability matrix, final report compilation; also owns data retention and audit test cases
Saad Imam	27079	Unit testing and Authentication / User Management test cases
Imad Raza	26953	Booking lifecycle test cases , concurrency/conflict tests, PO approval flow
Abdul Wasay Imran	27126	Room & Building management, E2E frontend tests, performance/load testing, security test cases

10.2 Test Environment and Tools

Category	Details
Operating Systems	Windows 10/11, CachyOS 6.19.12-1
Browsers	Google Chrome (latest stable), Mozilla Firefox (latest stable)
Frontend	React 18 + Vite
Backend	NestJS (Node.js)
Database	Supabase PostgreSQL (test instance, isolated from production)
Test Automation	Selenium WebDriver 4
API Testing	Postman v10
Test Management	TestRail
Defect Tracking	JIRA

10.3 Defect Severity Classification

Level	Label	Definition
S1	Critical	System crash, data loss, complete feature failure, or security breach
S2	Major	Core functionality broken; no acceptable workaround
S3	Minor	Feature works but with noticeable issues; simple workaround exists
S4	Trivial	Cosmetic issues, typos, or minor UI inconsistencies

11 Test Deliverables

The following artefacts will be produced as outputs of the testing effort:

1. **Test Plan Document** (this document)
2. **Test Cases:** covering all in-scope features with preconditions, steps, expected results, and actual results
3. **Test Data Set:** seed scripts containing test users, rooms, buildings, and booking records
4. **Defect Reports:** all defects logged with severity, priority, steps to reproduce, and fix status
5. **Test Execution Report:** pass/fail metrics per module, produced at the end of each execution cycle
6. **Test Summary Report:** final report covering total tests run, defect metrics, coverage achieved, deviations from plan, and release recommendation

12 Test Cases

This section outlines the high-level test cases designed to validate the functionality, reliability, and security of the IBA Campus Facility Booking System. Each test case is mapped to a feature in the SRS and is assigned a priority to guide the test execution schedule. These test cases will be further detailed in specific Test Case Specification documents with step-by-step procedures and explicit expected results.

12.1 Authentication & Authorisation

This set of tests ensures that the login functionality is secure and that user roles are correctly enforced across the application.

TC ID	Description	Type	Priority
TC-AUTH-001	A registered student logs in with a valid ERP and password and is successfully redirected to the student dashboard.	Functional	High
TC-AUTH-002	A registered Program Office (PO) member logs in with a valid username and password and is successfully redirected to the PO dashboard.	Functional	High
TC-AUTH-003	A user attempts to log in with an invalid ERP/username or an incorrect password. The system displays a clear, non-specific error message (e.g., “Invalid credentials”) and does not grant access.	Negative	High
TC-AUTH-004	A non-authenticated user attempts to directly access a protected URL (e.g., /dashboard). The system blocks access and redirects them to the login page.	Security	High
TC-AUTH-005	A logged-in student attempts to access a PO-only URL (e.g., /admin/pending-requests). The system prevents access and shows an “Unauthorized” or similar error.	Security	High
TC-AUTH-006	A logged-in PO member attempts to access an Admin-only URL (e.g., /admin/add-user). The system prevents access.	Security	High
TC-AUTH-007	User logs out. Verify that the session/token is destroyed and they cannot use the browser’s back button to view protected pages.	Security	Medium

Table 9: Authentication & Authorisation Test Cases

12.2 Room Booking Request

These tests focus on the core student workflow of finding and requesting a facility booking, including all business rule validations.

TC ID	Description	Type	Priority
TC-BOOK-001	(Happy Path) Student successfully submits a booking request for an available room during valid hours (e.g., Wednesday at 2:00 PM) and receives a confirmation message. The request appears in their dashboard with “Pending” status.	Functional	High
TC-BOOK-002	Student attempts to request a time slot that is already booked by another user. The system displays an error message indicating the conflict and does not submit the request.	Functional	High
TC-BOOK-003	Student attempts to request a room that has been disabled by an Admin for the selected date. The system displays an error message indicating the room is unavailable/disabled.	Functional	High
TC-BOOK-004	(Boundary) Student attempts to book a time slot exactly at 9:00 AM. The request is accepted.	Boundary	Medium
TC-BOOK-005	(Boundary) Student attempts to book a time slot ending exactly at 6:00 PM. The request is accepted.	Boundary	Medium
TC-BOOK-006	(Boundary) Student attempts to book a time slot starting at 8:59 AM. The request is rejected.	Boundary	Medium
TC-BOOK-007	(Boundary) Student attempts to book a time slot starting at 6:00 PM. The request is rejected.	Boundary	Medium
TC-BOOK-008	Student attempts to submit a booking request without filling in the mandatory “purpose” field. The system shows a validation error and prevents submission.	Negative	Medium
TC-BOOK-009	(Concurrency) Two different students attempt to submit a request for the exact same room and time slot simultaneously. The system correctly accepts the first request and rejects the second as a conflict.	Concurrency	High

Table 10: Room Booking Request Test Cases

12.3 Booking Management — Program Office

These tests validate the Program Office’s ability to manage incoming requests efficiently and accurately.

TC ID	Description	Type	Priority
TC-PO-001	PO member logs in and views a correctly populated list of all “Pending” booking requests on their dashboard.	Functional	High
TC-PO-002	PO approves a pending request. The booking status is updated to “Approved,” the room schedule is updated to show it as booked, and the student receives a notification.	Functional	High
TC-PO-003	PO rejects a pending request. The booking status is updated to “Rejected,” the time slot remains available for booking, and the student receives a notification.	Functional	High
TC-PO-004	(Conflict Resolution) Two students (A and B) have pending requests for the same time slot. The PO approves Student A’s request. Verify that Student B’s request is automatically rejected or flagged as a conflict, preventing a double booking.	Functional	High

Table 11: Booking Management (PO) Test Cases

12.4 Booking Cancellation

These tests ensure that bookings can be cancelled by authorised users according to the business rules.

TC ID	Description	Type	Priority
TC-CANCEL-001	A student cancels their own pending booking. The request is removed from the PO’s queue and the slot remains available.	Functional	Medium
TC-CANCEL-002	A student cancels their own approved booking well before the start time. The booking status changes to “Cancelled” and the time slot becomes available again.	Functional	High

TC ID	Description	Type	Priority
TC-CANCEL-003	(Boundary) A user attempts to cancel an approved booking 1 minute before its scheduled start time. The cancellation is successful.	Boundary	Medium
TC-CANCEL-004	(Boundary) A user attempts to cancel an approved booking 1 minute after its scheduled start time. The system prevents the cancellation.	Boundary	High
TC-CANCEL-005	A PO member successfully cancels a student's approved booking. The slot becomes available and the student is notified.	Functional	Medium

Table 12: Booking Cancellation Test Cases

12.5 Admin System Management

These tests cover the administrative functions for managing system data such as users, rooms, and buildings.

TC ID	Description	Type	Priority
TC-ADMIN-001	Admin successfully adds a new student account using a unique ERP.	Functional	High
TC-ADMIN-002	Admin attempts to add a new student with an ERP that already exists. The system shows a unique constraint error and does not create the account.	Negative	Medium
TC-ADMIN-003	Admin successfully adds a new PO member account with a unique username.	Functional	High
TC-ADMIN-004	Admin successfully adds a new building with a unique name.	Functional	Medium
TC-ADMIN-005	Admin successfully adds a new room, assigning it to an existing building.	Functional	Medium
TC-ADMIN-006	Admin successfully disables an existing room for a specified period. Verify that this room does not appear as available for booking during that period.	Functional	High
TC-ADMIN-007	Admin re-enables a previously disabled room. Verify that the room becomes available for booking again.	Functional	Medium

Table 13: Admin System Management Test Cases

12.6 Data Integrity

To ensure data is stored reliably.

TC ID	Description	Type	Priority
TC-DATA-001	After a booking is created, approved, and cancelled, verify that an accurate audit trail of these status changes is maintained in the database for future reference.	Functional	Medium

Table 14: Performance & Data Integrity Test Cases